Laboratório 2

Alessandro Silva - RA: 236928

```
import polars as pl

PATH_BASICS     = "title.basics0.tsv"
PATH_RATINGS    = "title.ratings.tsv"
PATH_PRINCIPALS = "title.principals0.tsv"

def le_tsv(path):
    return pl.scan_csv(
        path,
        separator="\t",
        null_values=["\\N"],
        quote_char=None,
        infer_schema_length=10_000,)

basics     = le_tsv(PATH_BASICS)
ratings    = le_tsv(PATH_RATINGS)
principals = le_tsv(PATH_PRINCIPALS)
```

```
filmes = (
    basics
    .filter(pl.col("titleType") == "movie")
    .join(ratings, on="tconst", how="inner")
)
```

Questão 1

```
top5 = (
    filmes
    .select("tconst", "primaryTitle", "startYear", "averageRating", "numVotes")
    .sort(["averageRating", "numVotes"], descending=[True, True])
    .head(5)
    .collect(engine="streaming")
)

top5
```

```

```

```
shape: (5, 5)
┌────────────┬──────────────────────────────┬────────────┬─────────────────┬────────────┐
│ tconst     ┆ primaryTitle                 ┆ startYear  ┆ averageRating   ┆ numVotes   │
│            │                              │            │                 │            │
│ ---        ┆ ---                          ┆ ---        ┆ ---             ┆ ---        │
│ str        ┆ str                          ┆ i64        ┆ f64             ┆ i64        │
╞════════════╪══════════════════════════════╪════════════╪═════════════════╪════════════╡
│ tt33075815 ┆ Kaveri                       ┆ 2024       ┆ 10.0            ┆ 1023       │
│ tt27815015 ┆ Kurukku                      ┆ 2024       ┆ 10.0            ┆ 451        │
│ tt28450376 ┆ Jedal Dar Omghe 30 Metri     ┆ 2009       ┆ 10.0            ┆ 142        │
│ tt27859713 ┆ Sargashte                    ┆ 2016       ┆ 10.0            ┆ 134        │
│ tt33507675 ┆ Gorgeous Rascal              ┆ 2024       ┆ 10.0            ┆ 115        │
└────────────┴──────────────────────────────┴────────────┴─────────────────┴────────────┘
```

Questão 2

```
generos = (
    filmes
    .filter(pl.col("averageRating") > 8)
    .select("tconst", "genres")
    .drop_nulls("genres")
    .with_columns(pl.col("genres").str.split(","))
    .explode("genres")
    .group_by("genres")
    .agg(pl.len().alias("n_filmes"))
    .sort("n_filmes", descending=True)
    .collect(engine="streaming")
)

generos.head(10)
```

```
shape: (10, 2)
┌──────────────┬──────────┐
│ genres       ┆ n_filmes │
│ ---          ┆ ---      │
│ str          ┆ u32      │
╞══════════════╪══════════╡
│ Documentary  ┆ 10564    │
│ Drama        ┆ 8114     │
│ Comedy       ┆ 3186     │
│ Biography    ┆ 1395     │
│ Romance      ┆ 1363     │
│ Action       ┆ 1355     │
│ Music        ┆ 1248     │
│ Thriller     ┆ 1184     │
│ History      ┆ 1030     │
```

```
| Crime       ¦ 977       |
└─────────────┴─────────────┘
```

```
generos.head(1)
```

```
shape: (1, 2)
┌─────────────┬─────────────┐
| genres      ¦ n_filmes |
| ---         ¦ ---       |
| str         ¦ u32       |
╞═════════════╪═════════════╡
| Documentary ¦ 10564     |
└─────────────┴─────────────┘
```

Questão 3

```
filmes_bons = (
    filmes
    .filter(pl.col("averageRating") > 7.5)
    .select("tconst")
)

top3_atores = (
    principals
    .filter(pl.col("category").is_in(["actor", "actress"]))
    .select("tconst", "nconst", "category")
    .join(filmes_bons, on="tconst", how="inner")
    .group_by("nconst")
    .agg(pl.col("tconst").n_unique().alias("n_filmes"))
    .sort("n_filmes", descending=True)
    .head(3)
    .collect(engine="streaming")
)

top3_atores
```

```
shape: (3, 2)
┌─────────────┬─────────────┐
| nconst      ¦ n_filmes |
| ---         ¦ ---       |
| str         ¦ u32       |
╞═════════════╪═════════════╡
| nm0004660 ¦ 199       |
| nm0595934 ¦ 155       |
```

```
| nm3183374 ¦ 124       |
└─────────────┴─────────────┘
```